第七章 半导体存储器

- 7.1 概述
- 7.2 只读存储器（ROM）
- 7.3 随机存储器（RAM）
- 7.4 存储器容量的扩展
- 7.5 用存储器实现组合逻辑函数

7.1 概述

半导体存储器是一种能存储大量二值信息（或称为二值的数据）的半导体器件。

分类：

1. 从存储功能上可分为：

只读存储器（Read Only Memory）

随机存储器（Random Access Memory）

2. 从制造工艺上可分为：

双极型

MOS型

7.1 概述

ROM

- **掩模ROM**：数据在制作时已经确定，无法更改。
- **可编程ROM（PROM）**：
  数据可由用户一次写入，但不能再修改。
- **可擦除的可编程ROM（EPROM）**：
  数据可多次擦写，灵活性更大。

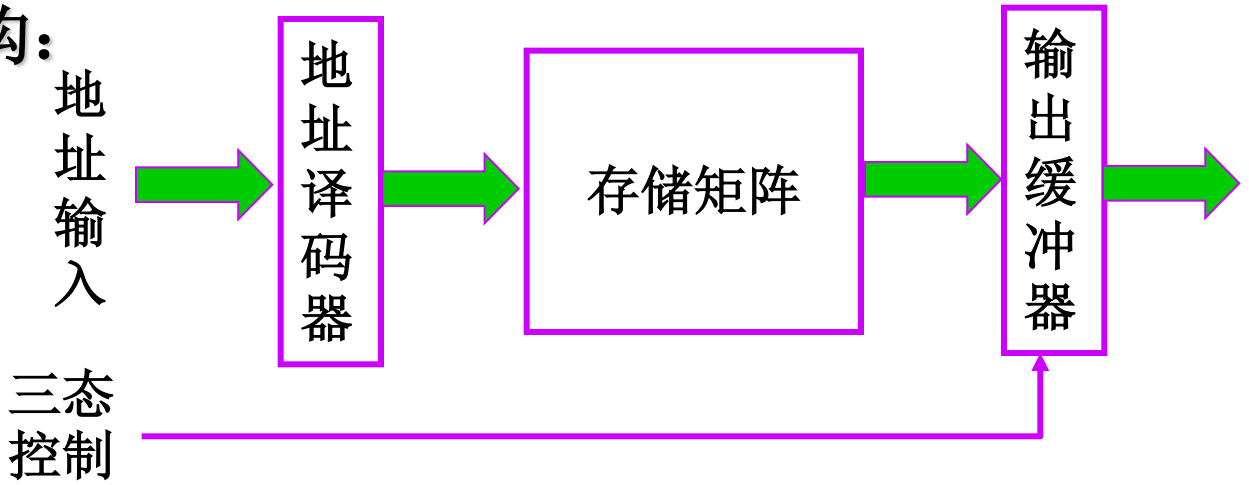
RAM

- **静态存储器（SRAM）**：存取速度快。
- **动态存储器（DRAM）**：集成度高 。

7.2 掩模只读存储器

7.2.1掩模只读存储器

掩模ROM在出厂时内部存储的数据就已经“固化”。ROM的电路结构：

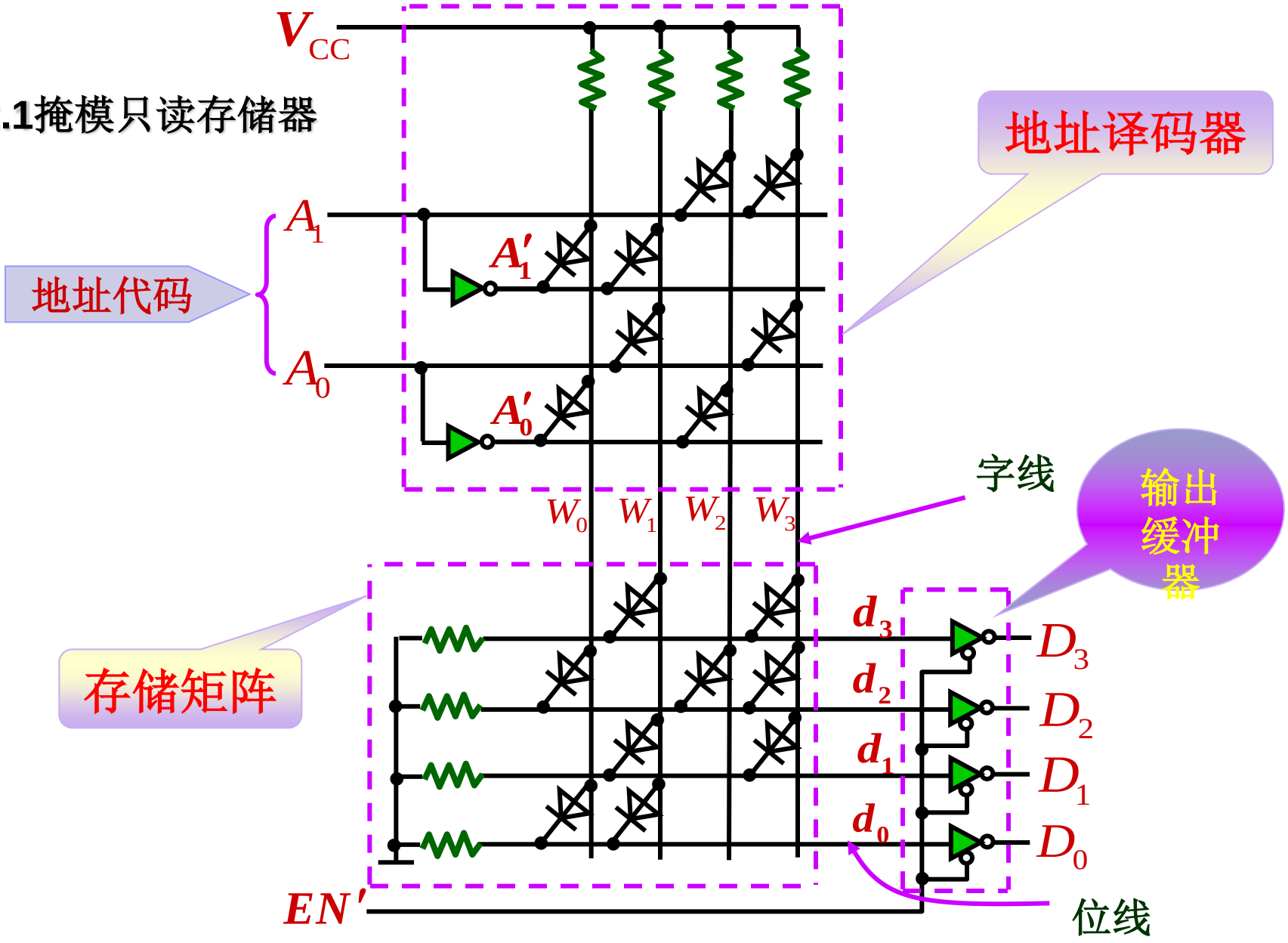


存储矩阵：由许多存储单元排列而成。每个单元能放1位二值代码（0或1）。每一个或一组存储单元有一个对应的地址代码。

地址译码器：将输入的地址代码译成相应的控制信号，利用这个控制信号从存储矩阵中把指定的单元选出，并把其中的数据送到输出缓冲器。

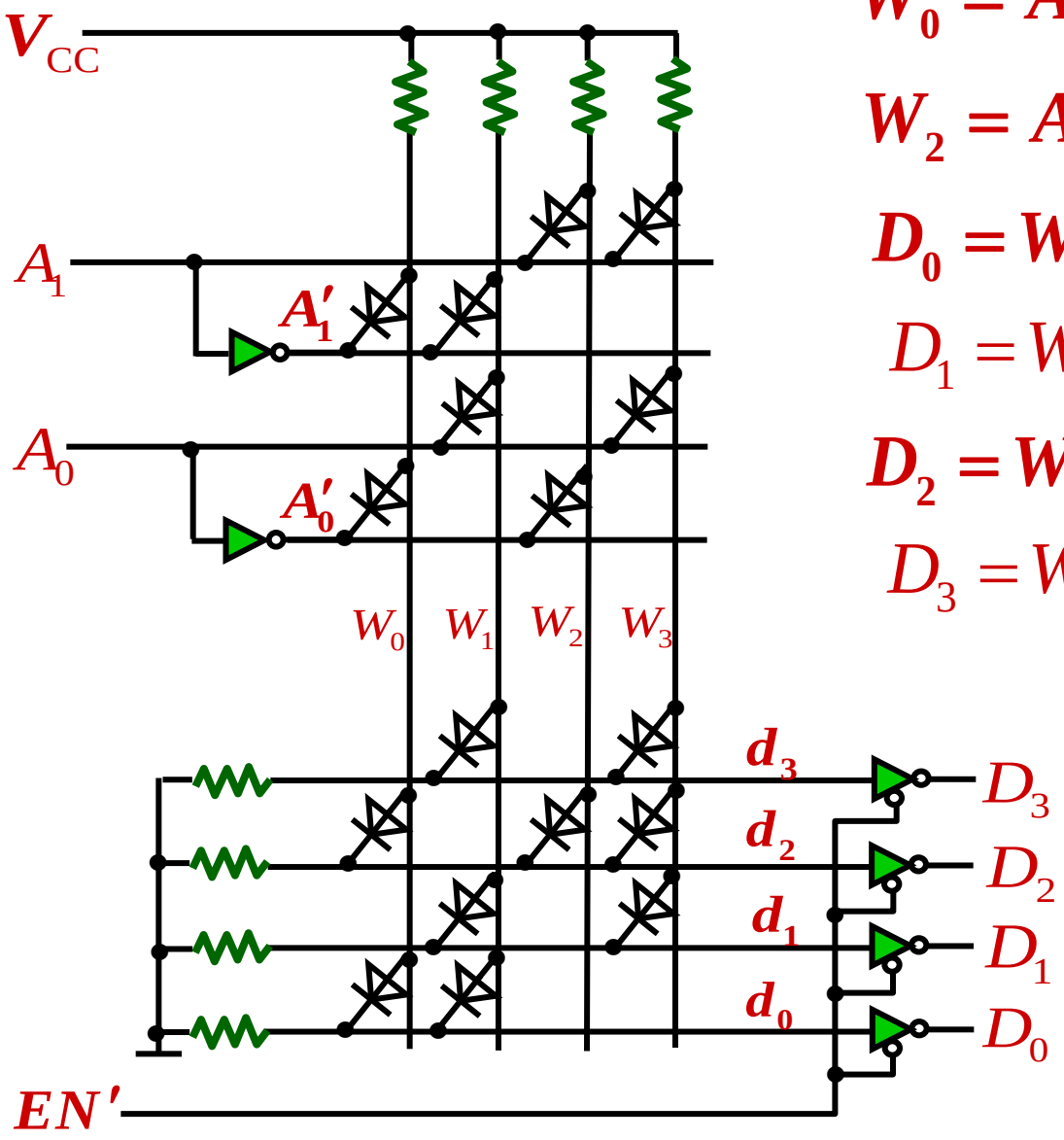
输出缓冲器：一是能提高存储器的带负载能力，二是实现对输出状态的三态控制，以便与系统总线连接。

7.2.1掩模只读存储器



二极管ROM的电路结构图

7.2.1掩模只读存储器



二极管ROM的电路结构图

$$W_0 = A_1'A_0' \qquad W_1 = A_1'A_0$$

$$W_2 = A_1A_0' \qquad W_3 = A_1A_0$$

$$D_0 = W_1 + W_0 = A_1'$$

$$D_1 = W_1 + W_3 = A_0$$

$$D_2 = W_0 + W_2 + W_3 = A_1 + A_0'$$

$$D_3 = W_1 + W_3 = A_0$$

ROM中的数据表

地址		数据			
A_1	A_0	D_3	D_2	D_1	D_0
0	0	0	1	0	1
0	1	1	0	1	1
1	0	0	1	0	0
1	1	1	1	1	0

7.2 掩模只读存储器

7.2.2 可编程只读存储器（PROM）

总体结构与掩模ROM一样，但出厂时已经在存储矩阵的所有交叉点上全部制作了存储元件，相当于在所有存储单元中都存入了 1。

写入数据时，设法将需要存入0的那些存储单元上的熔丝烧断就可以。

PROM的内容只能写入一次。

7.2 掩模只读存储器

7.2.3 可擦除的可编程只读存储器（EPROM）

1. EPROM（UVEPROM）

紫外线可擦除的可编程ROM（Ultra-Voilet Erasable Programmable Read-Only Memory）

早期存储单元中使用浮栅雪崩注入MOS管，目前多改用叠栅注入MOS管，

擦除操作复杂，擦除速度很慢。

2. E^2PROM

电信号可擦除的可编程ROM（Electrically Erasable Programmable Read-Only Memory）

存储单元中采用了浮栅隧道氧化层MOS管，

擦除和写入时需要加高电压脉冲，擦、写时间仍较长，

正常工作状态下，只能工作在读出状态。

7.2 掩模只读存储器

7.2.3 可擦除的可编程只读存储器（EPROM）

3. 快闪存储器（Flash Memory）

电信号可擦除的可编程ROM

即吸收了EPROM结构简单、编程可靠的优点，

又保留了E^2PROM用隧道效应擦除的快捷特性，

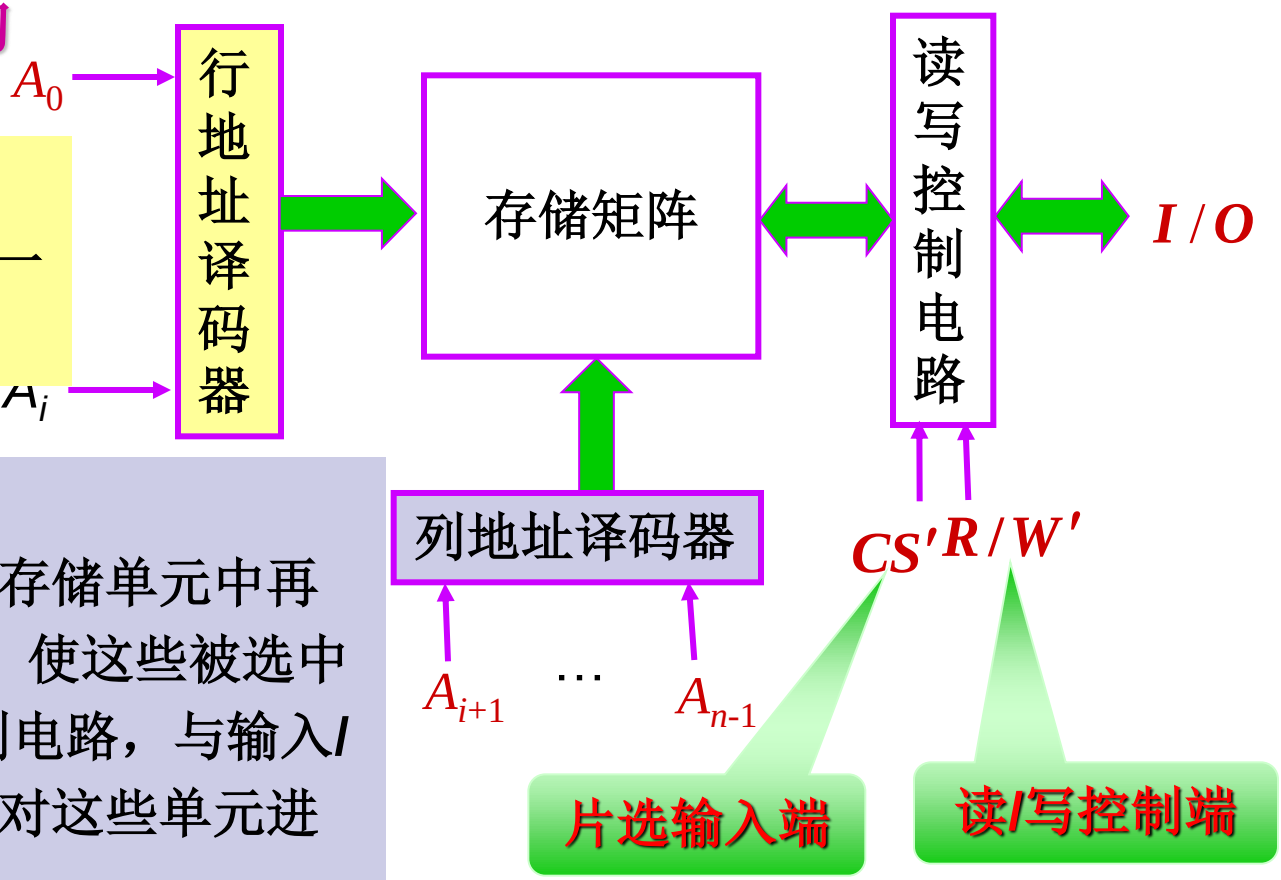
因此具有以下优点：

集成度高、容量大、成本低、使用方便。

7.3 随机存储器（RAM）

7.3.1 静态随机存储器（SRAM）

1. SRAM的结构



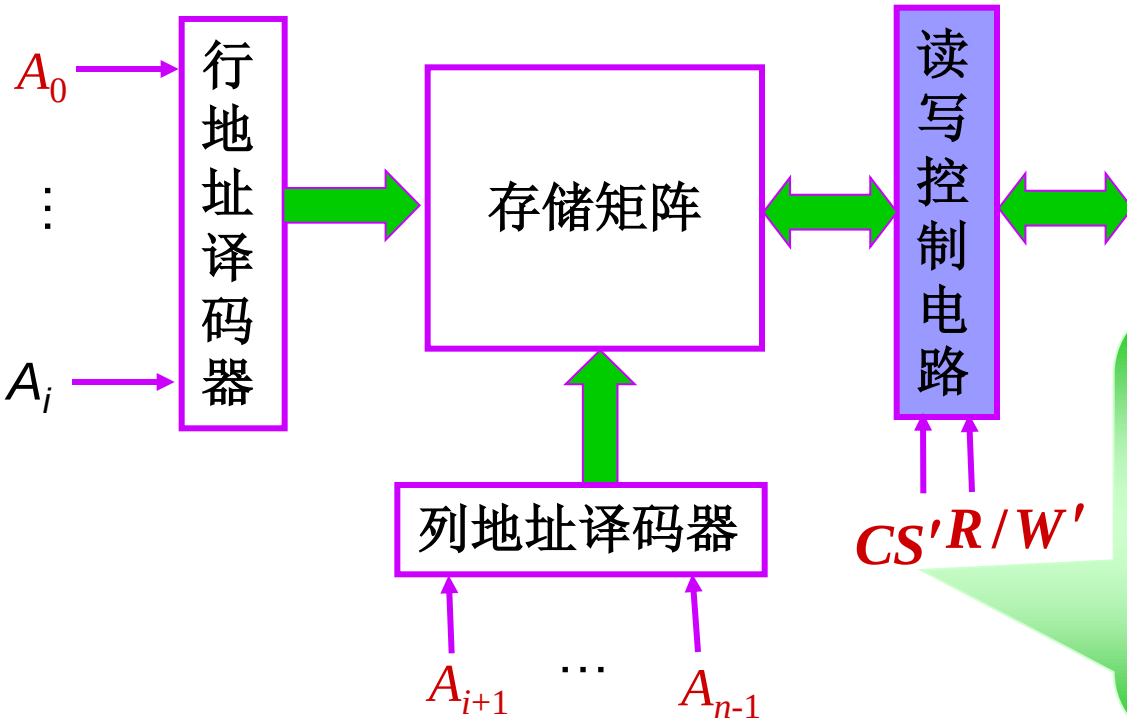
行地址译码器

从存储矩阵中选中一行存储单元；

列地址译码器

从字线选中的一行存储单元中再选1位（或几位），使这些被选中的单元经读/写控制电路，与输入/输出端接通，以便对这些单元进行读/写操作。

1. SRAM的结构



片选输入端：
片选输入信号为0时，RAM为正常工作状态。
片选输入信号为1时，不能对RAM进行读/写操作。

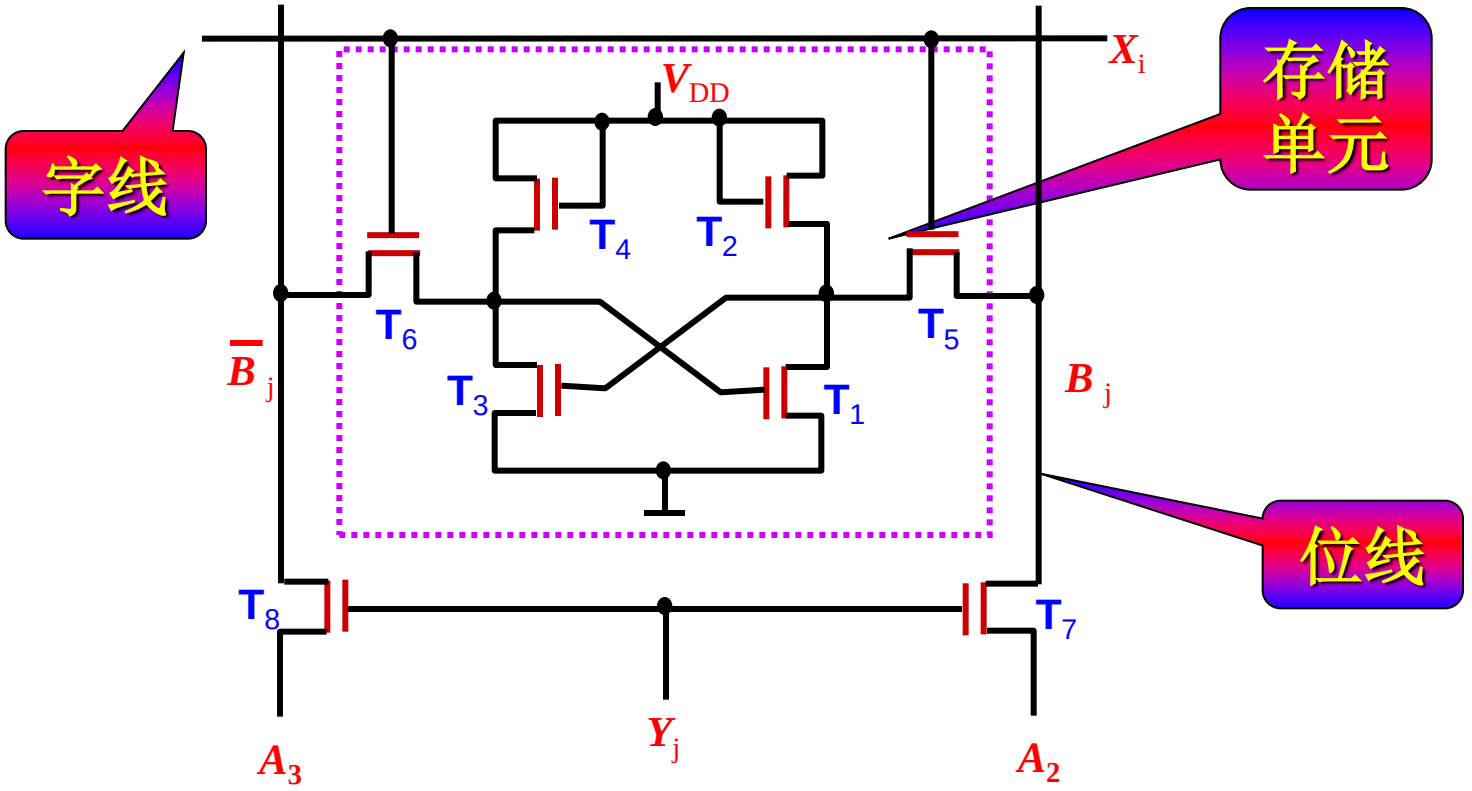
读/写控制电路：
用于对电路的工作状态进行控制。当读/写控制信号为1时，执行读操作，将存储单元里的数据送到输入/输出端上。
当读/写控制信号为0时，执行写操作，加到输入/输出端上的数据被写到存储单元中。

7.3 随机存储器（RAM）

7.3.1 静态随机存储器（SRAM）

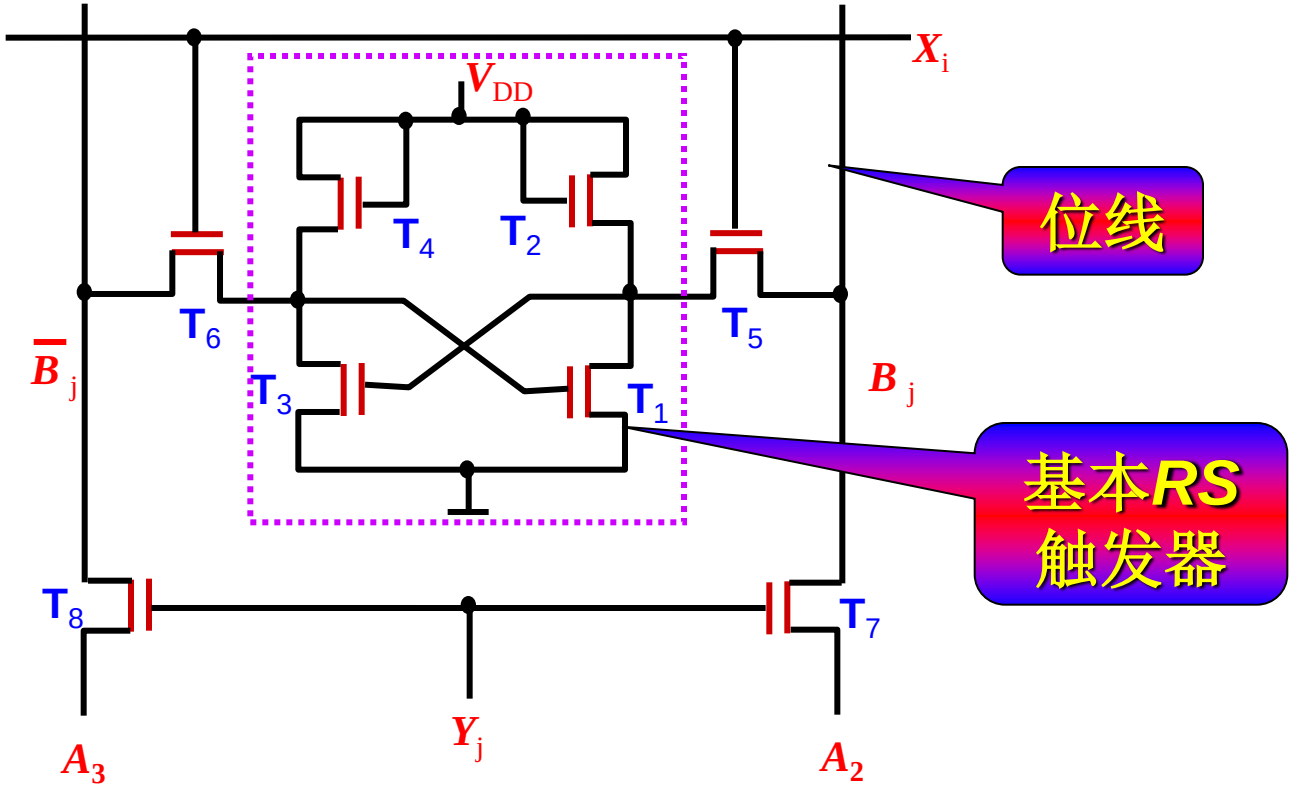
2. SRAM的静态存储单元

静态存储单元是在静态触发器的基础上附加门控管而构成的



六管NMOS静态存储单元

2. SRAM的静态存储单元

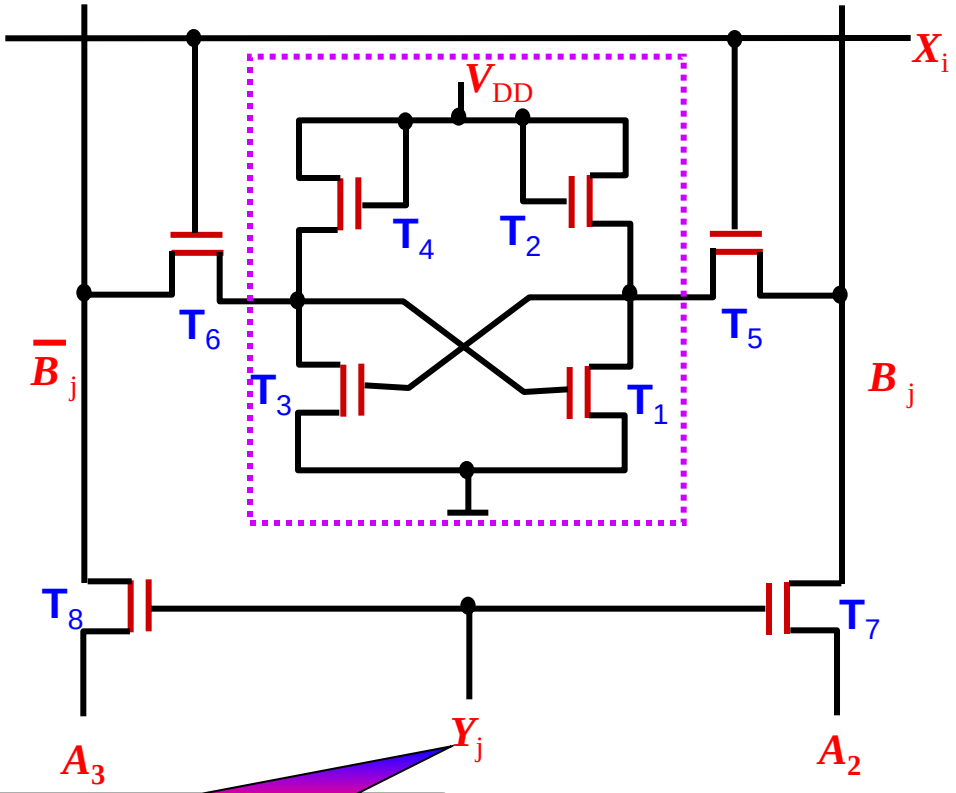


T_5和T_6是门控管，作模拟开关（X_i决定开关状态），用以控制触发器的输出和位线之间的关系。

X_i = 1时T_5、T_6导通，触发器与位线接通。

X_i = 0时T_5、T_6截止，触发器与位线断开。

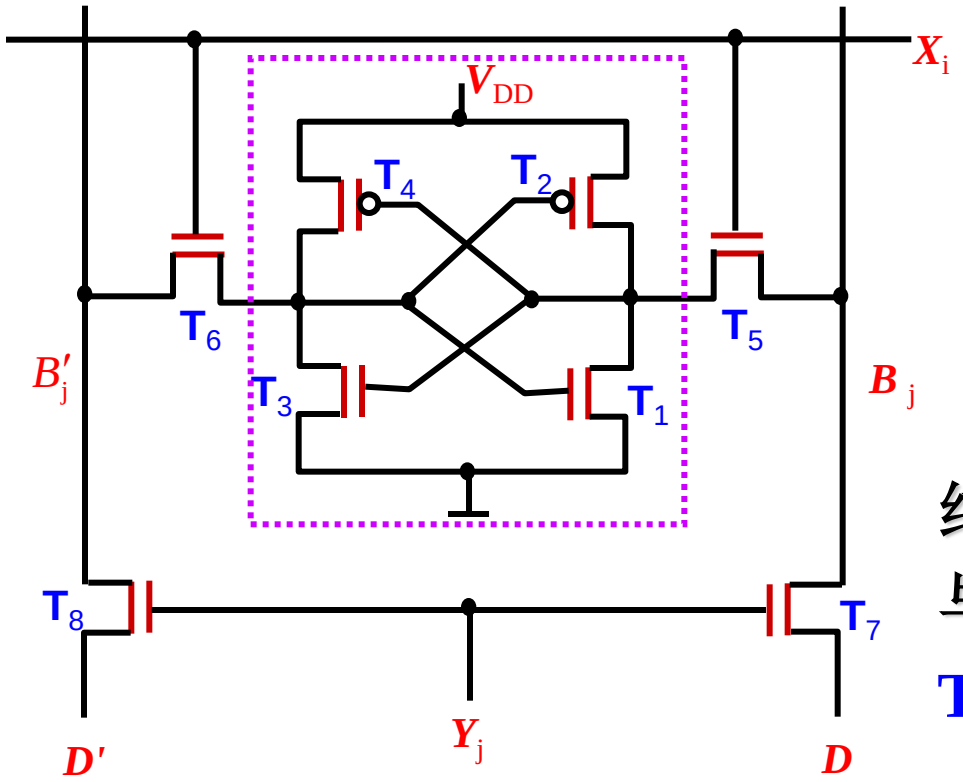
2. SRAM的静态存储单元



T_7、T_8是每一列存储单元公用的门控管，用于和读/写缓冲放大器之间的连接。

Y_j = 1时T_7、T_8导通，Y_j = 0时T_7、T_8截止。

2. SRAM的静态存储单元



六管CMOS静态存储单元

结构形式和工作原理，
与六管NMOS存储单元相仿，
T_2、T_4是P沟道MOS管，

采用CMOS工艺的SRAM正常工作时功耗很低，能在降低电源电压的状态下保存数据。

7.3 随机存储器（RAM）

*7.3.2 动态随机存储器（DRAM）

1. DRAM的动态存储单元

RAM的动态存储单元是利用MOS管栅极电容可以存储电荷的原理制成的。

存储单元的结构能做得非常简单，普遍应用于大容量、高集成度的RAM中。由于栅极电容的容量很小（通常仅为几皮法），而漏电流又不可能绝对等于零，所以电荷保存的时间有限。

为了及时补充漏掉的电荷以避免存储的信号丢失，必须定时给栅极电容补充电荷，通常将这种操作称为刷新或再生。因此，DRAM工作时必须辅以必要的刷新控制电路，同时也使操作复杂化了。

7.3 随机存储器（RAM）

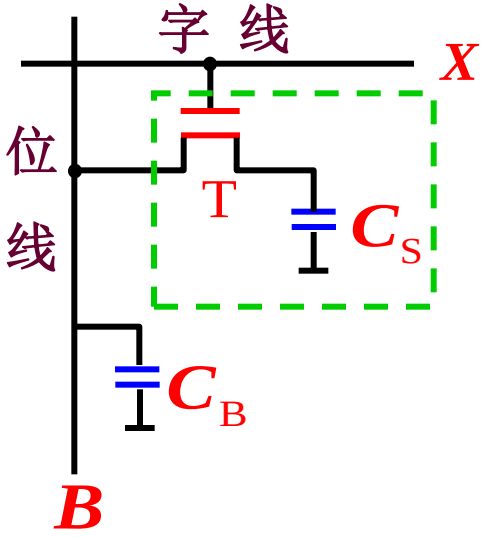
*7.3.2 动态随机存储器（DRAM）

1. DRAM的动态存储单元

早期采用的动态存储单元为四管电路或三管电路。这两种电路的优点是外围控制电路比较简单，读出信号也比较大。

缺点是电路结构仍不够简单，不利于提高集成度。

单管动态存储单元是所有存储单元中电路结构最简单的一种。是目前所有大容量DRAM首选的存储单元。



单管动态MOS存储单元

在进行写操作时，字线给出高电平，使T导通，位线上的数据便通过T被存入C_S中。

在进行读操作时，字线同样给出高电平，使T导通，C_S经T向位线上的电容C_B提供电荷，使位线获得读出的信号电平。

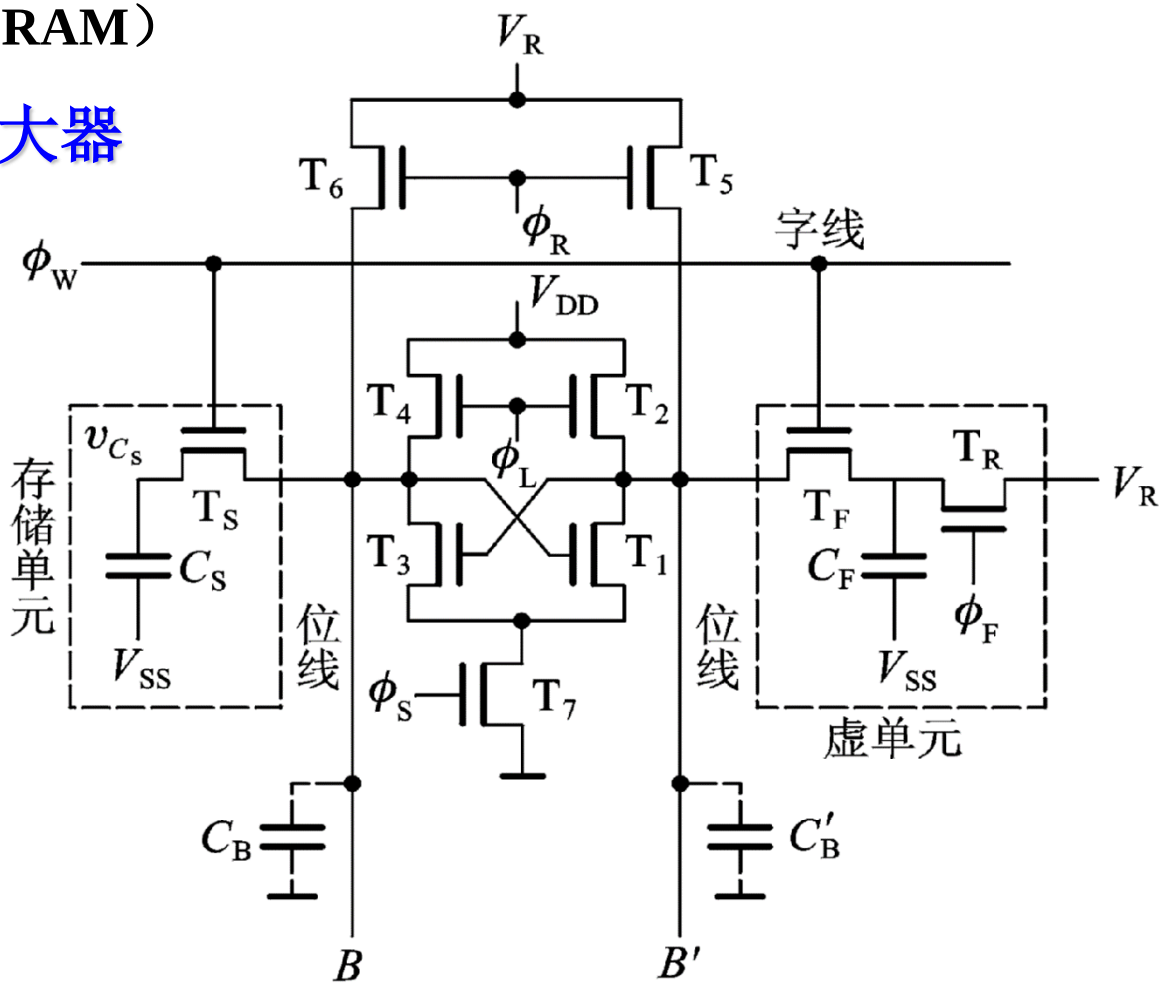
7.3 随机存储器（RAM）

***7.3.2 动态随机存储器（DRAM）**

2. 灵敏度恢复/读出放大器

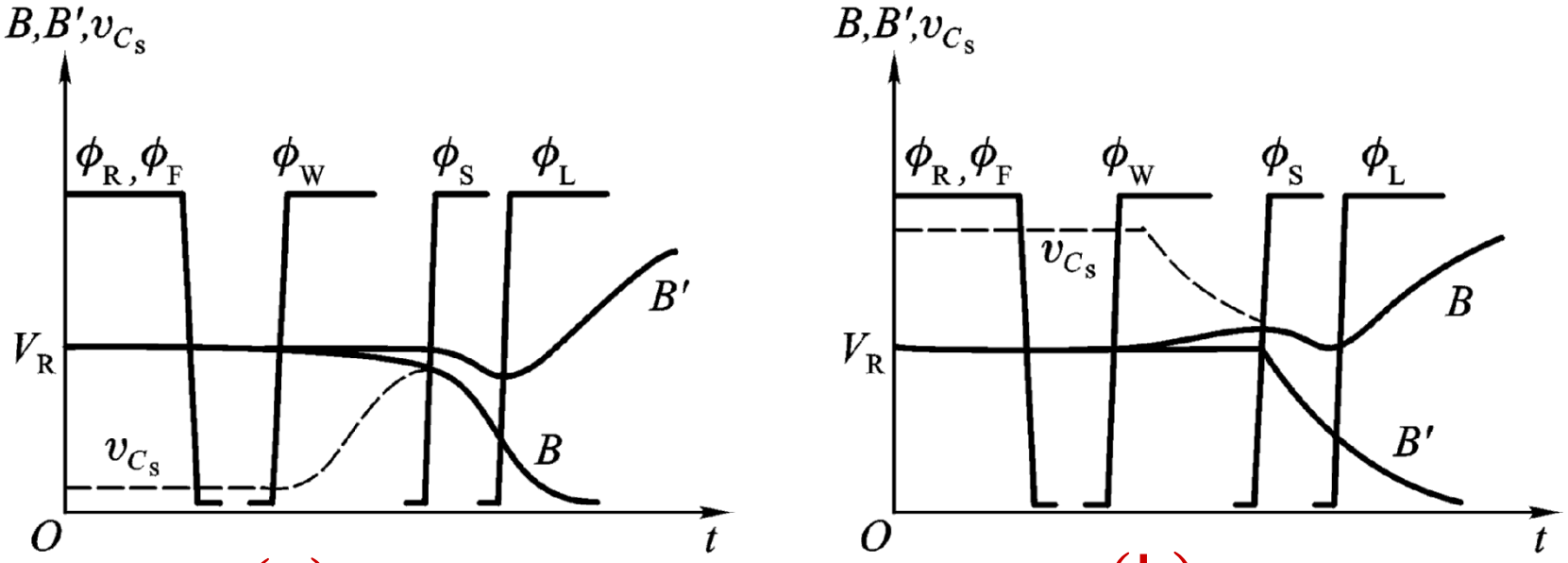
DRAM中的单管动态存储单元也是按行、列排成矩阵式结构，

并且在每根位线上接有灵敏度恢复/读出放大器。



DRAM中的灵敏度恢复/读出放大器

2. 灵敏度恢复/读出放大器



灵敏度恢复/读出放大器的读出过程

（a）读出0的情况　　（b）读出1的情况

使用了灵敏度恢复/读出放大器之后，在每次读出数据的同时也完成了对存储单元原来所存数据的刷新。

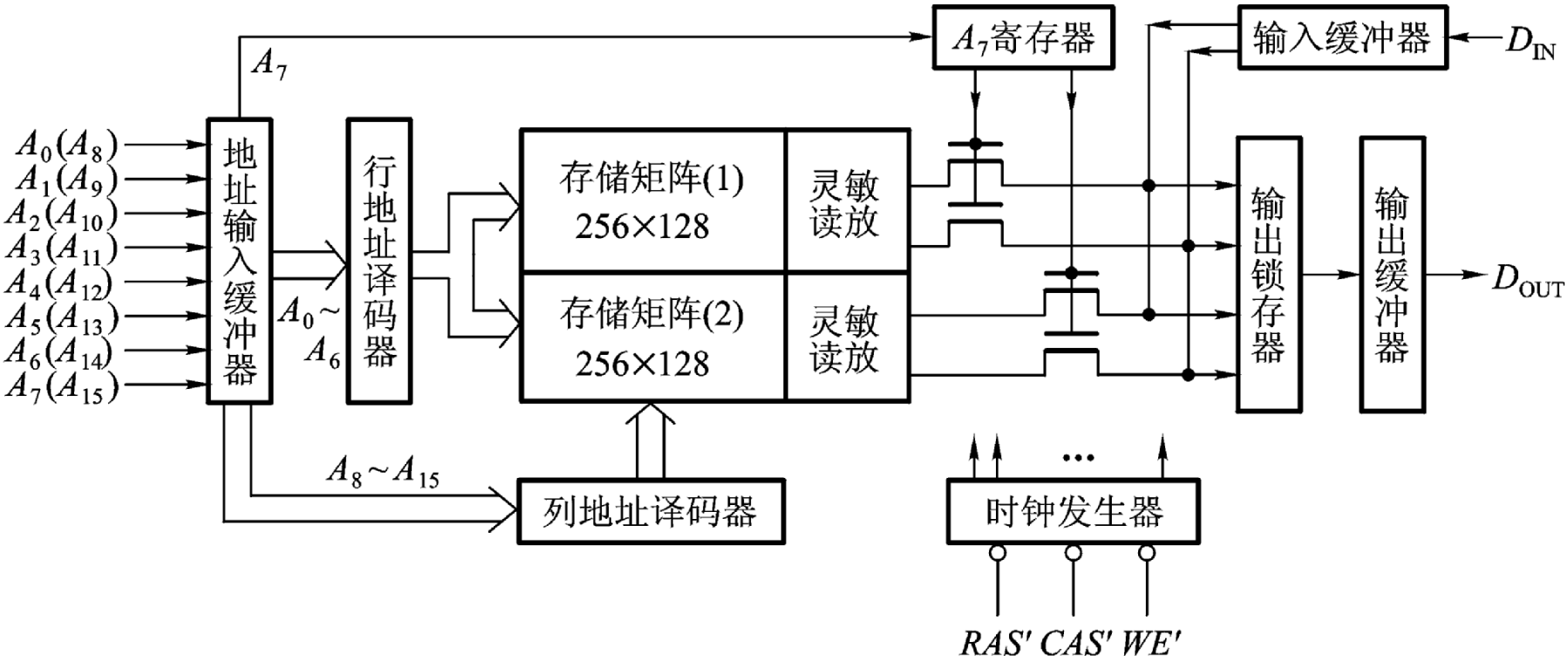
因此，DRAM中的刷新操作是通过按行依次执行一次操作来实现的。刷新时输出被置成高阻态。

7.3 随机存储器（RAM）

***7.3.2 动态随机存储器（DRAM）**

3. DRAM的总体结构

为了提高集成度的同时减少器件引脚的数目，目前的大容量DRAM多半都采用1位输入、1位输出和地址分时输入的方式。

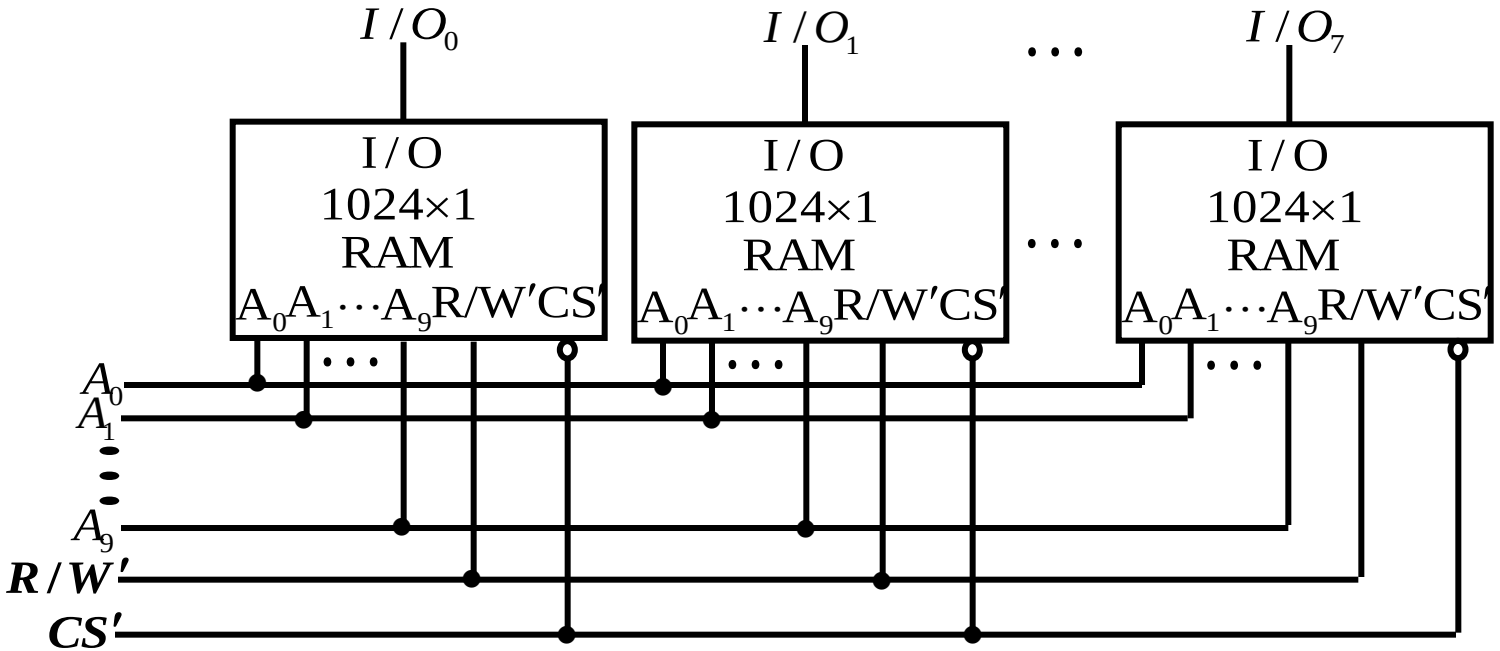


DRAM的总体结构框图

7.4 存储器容量的扩展

7.4.1 位扩展方式

如果每一片**ROM**或**RAM**中的字数已够用，而每个字的位数不够用时，应采用位扩展的连接方式。

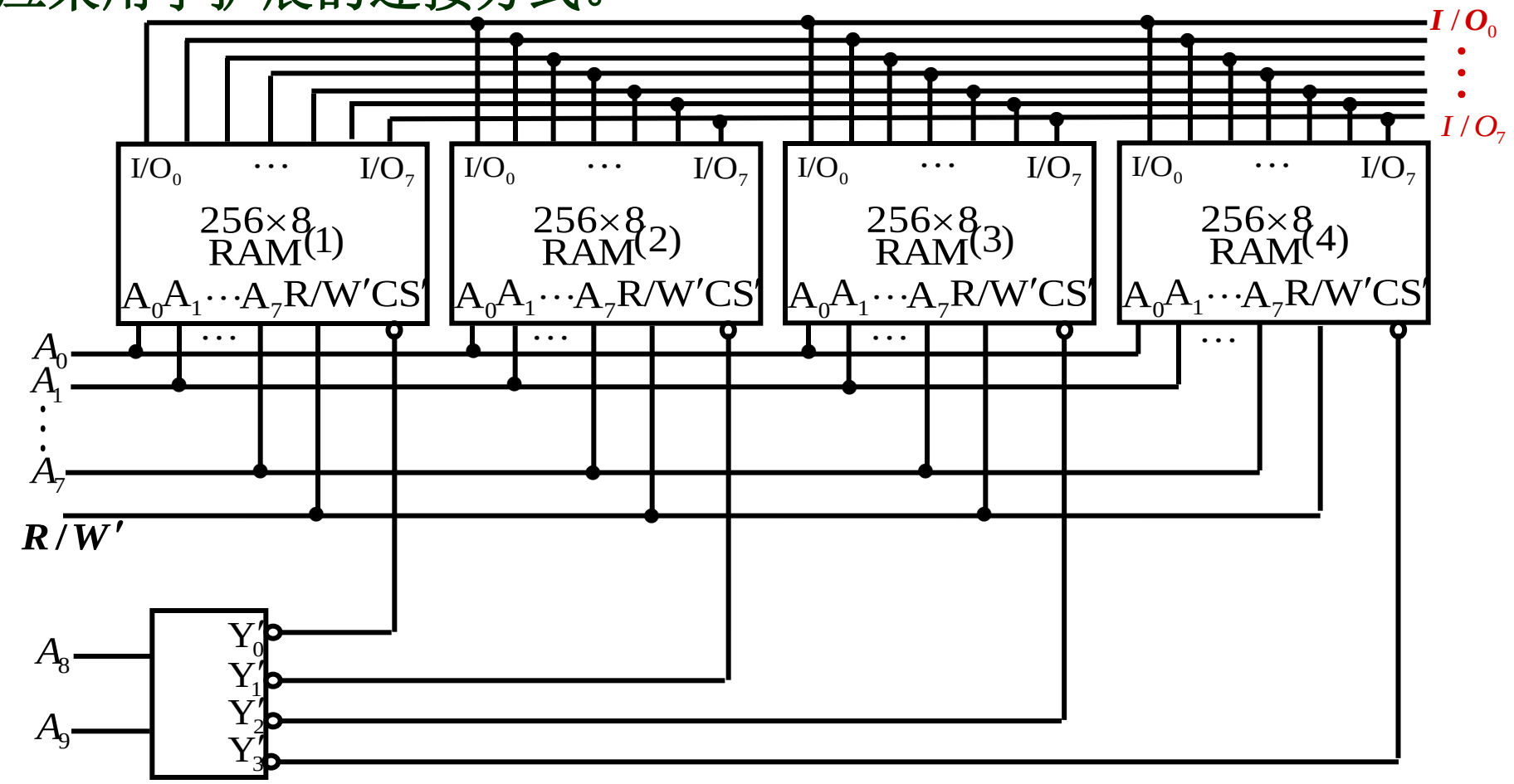


RAM的位扩展接法

7.4 存储器容量的扩展

7.4.1 字扩展方式

如果每一片**ROM**或**RAM**中的位数够用，而字数不够用时，应采用字扩展的连接方式。



7.4 存储器容量的扩展

7.4.1 字扩展方式

字扩展中各片RAM电路的地址分配

器件编号	A_9 A_8	Y_0' Y_1' Y_2' Y_3'	地址范围 A_9 A_8 A_7 A_6 A_5 A_4 A_3 A_2 A_1 A_0（等效十进制数）
RAM (1)	0 0	0 1 1 1	00 00000000 ~ 00 11111111 (0) (255)
RAM (2)	0 1	1 0 1 1	01 00000000 ~ 01 11111111 (256) (511)
RAM (3)	1 0	1 1 0 1	10 00000000 ~ 10 11111111 (512) (767)
RAM (4)	1 1	1 1 1 0	11 00000000 ~ 11 11111111 (768) (1023)

7.5 用存储器实现组合逻辑

存储器中的数据表

地 址		数 据			
A_1	A_0	D_3	D_2	D_1	D_0
0	0	1	0	0	1
0	1	0	1	1	1
1	0	1	1	1	0
1	1	0	1	0	1

从存储器的角度看，只要将逻辑函数的真值表事先存入ROM，便可用ROM实现该函数。如将输入地址A_1、A_0看成两个输入逻辑变量，而将数据输出D_3、D_2、D_1、D_0看成一组输出逻辑变量，则$D3$、D_2、D_1、D_0就是A_1、A_0的一组逻辑函数。

用ROM实现逻辑函数一般按以下步骤进行：

① 根据逻辑函数的输入、输出变量数目，确定ROM的容量，选择合适的ROM。

② 写出逻辑函数的最小项表达式，画出ROM的阵列图。

③ 根据阵列图对ROM进行编程。

一、利用ROM实现组合逻辑函数

例：试用PROM产生一组多输出逻辑函数

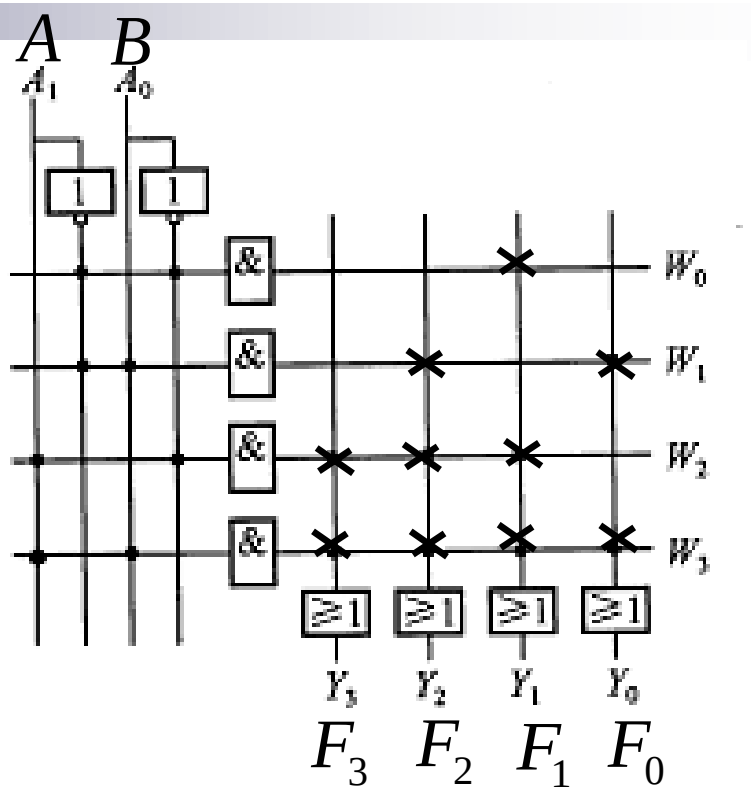
$$F_0 = \overline{A}B + AB \qquad F_1 = \overline{A}\cdot\overline{B} + A$$

$$F_2 = \overline{A}B + A \qquad F_3 = A$$

组合电路真值表　　存储器数据表

N个输入　　N个地址

m个输出　　m位数据



组合电路真值表

输入		输出			
A	B	F_3	F_2	F_1	F_0
0	0	0	0	1	0
0	1	0	1	0	1
1	0	1	1	1	0
1	1	1	1	1	1

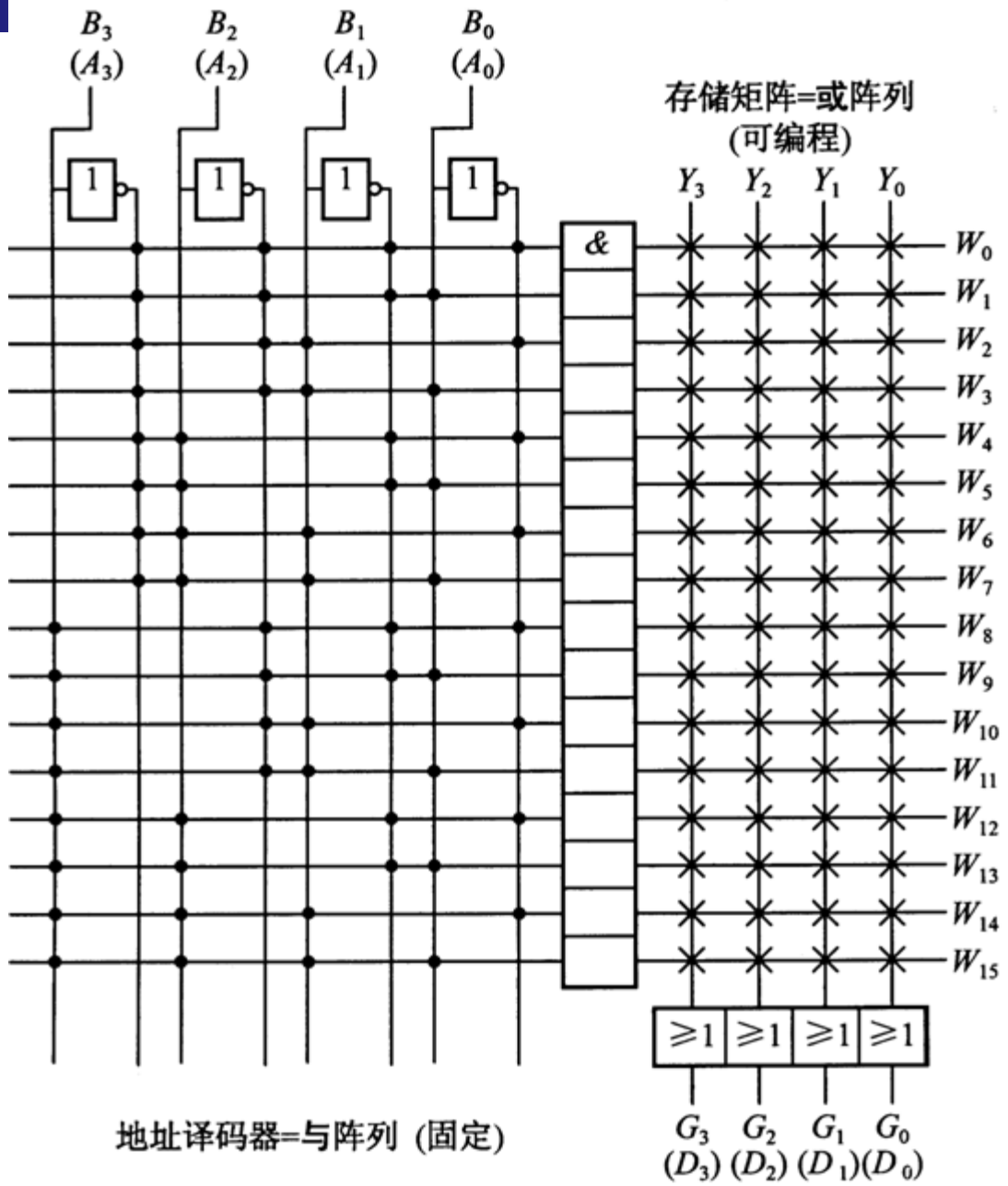
存储器数据表

地址		内容			
A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	1	0
0	1	0	1	0	1
1	0	1	1	1	0
1	1	1	1	1	1

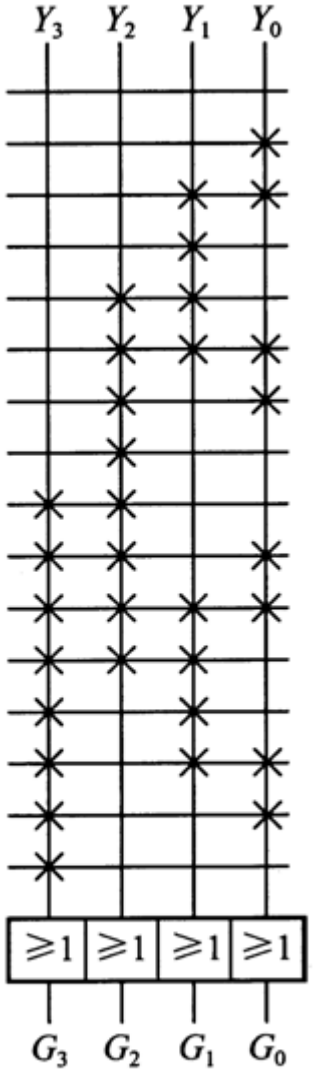
例 用PROM设计一个四位二进制码转换为格雷码的代码转换电路。

解： 输入是四位自然二进制码B_3~B_0，输出是四位循环码G_3~G_0，故选$2^4\times4$的PROM。

字	二进制码				循环码			
	B_3	B_2	B_1	B_0	G_3	G_2	G_1	G_0
W_0	0	0	0	0	0	0	0	0
W_1	0	0	0	1	0	0	0	1
W_2	0	0	1	0	0	0	1	1
W_3	0	0	1	1	0	0	1	0
W_4	0	1	0	0	0	1	1	0
W_5	0	1	0	1	0	1	1	1
W_6	0	1	1	0	0	1	0	1
W_7	0	1	1	1	0	1	0	0
W_8	1	0	0	0	1	1	0	0
W_9	1	0	0	1	1	1	0	1
W_{10}	1	0	1	0	1	1	1	1
W_{11}	1	0	1	1	1	1	1	0
W_{12}	1	1	0	0	1	0	1	0
W_{13}	1	1	0	1	1	0	1	1
W_{14}	1	1	1	0	1	0	0	1
W_{15}	1	1	1	1	1	0	0	0



(a) 未编程的 16×4 位 PROM



(b) 编程后的或阵列

例、用ROM实现组合逻辑函数

逻辑表达式

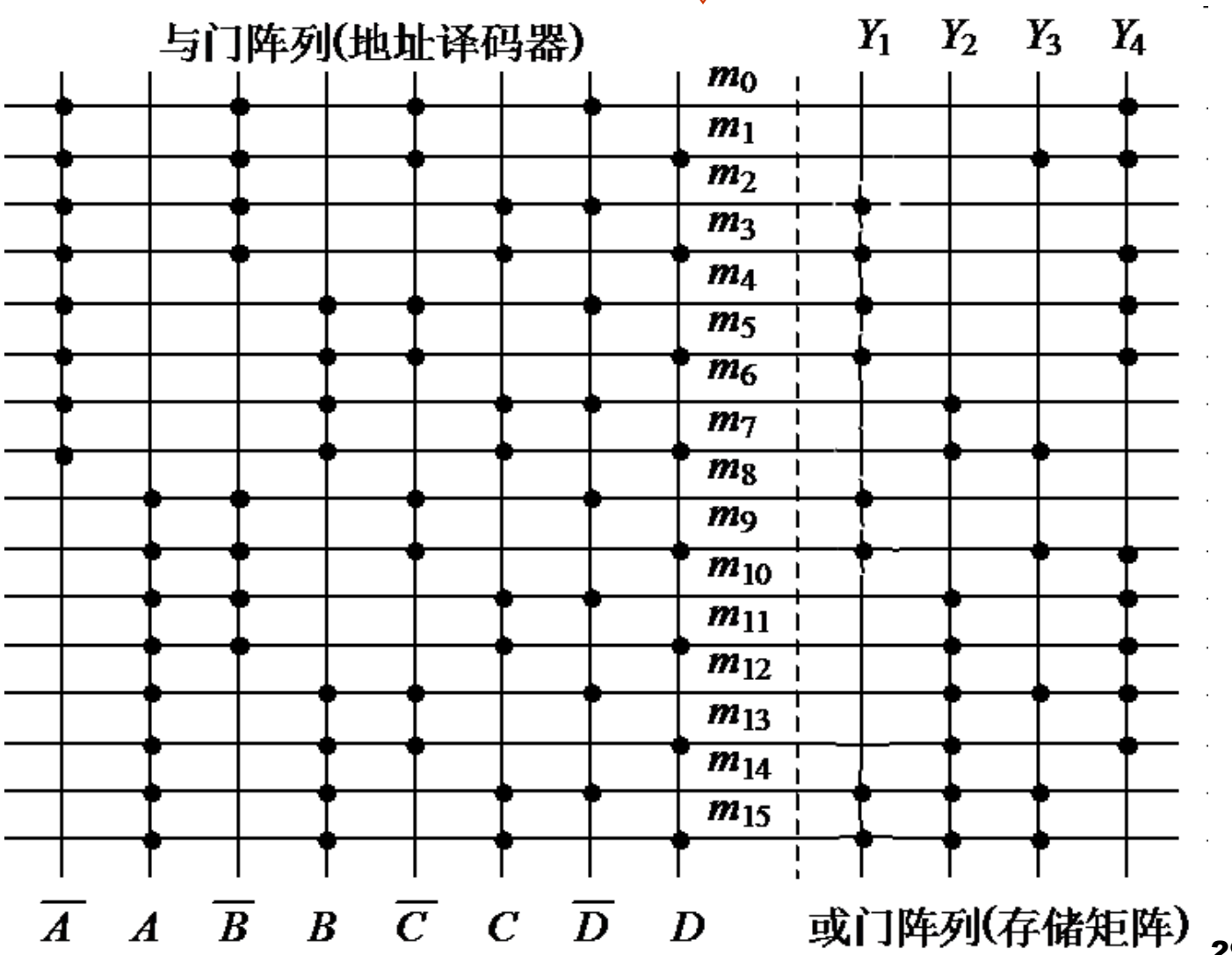
$$\begin{cases} Y_1 = A \oplus B \oplus C \\ Y_2 = AB + AC + BC \\ Y_3 = AB\overline{D} + BCD + \overline{B}\,\overline{C}D \\ Y_4 = \overline{A}\,\overline{C} + B\overline{C} + \overline{B}D + A\overline{B}C \end{cases}$$

按***A***、***B***、***C***、***D***排列变量，并将Y_1、Y_2扩展成为4变量的逻辑函数。

真值表或最小项表达式

$$\begin{cases} Y_1 = \sum m(2,3,4,5,8,9,14,15) \\ Y_2 = \sum m(6,7,10,11,12,13,14,15) \\ Y_3 = \sum m(1,7,9,12,14,15) \\ Y_4 = \sum m(0,1,3,4,5,9,10,11,12,13) \end{cases}$$

选择ROM，画阵列图


与门阵列(地址译码器)
Y_1 Y_2 Y_3 Y_4
m_0
m_1
m_2
m_3
m_4
m_5
m_6
m_7
m_8
m_9
m_{10}
m_{11}
m_{12}
m_{13}
m_{14}
m_{15}
$\overline{A}$ A $\overline{B}$ B $\overline{C}$ C $\overline{D}$ D
或门阵列(存储矩阵)

二、用ROM作函数运算表

例 用ROM构成能实现函数$y=x^2$的运算表电路。

设x的取值范围为0～15的正整数，则对应的是4位二进制正整数，用$B=B_3B_2B_1B_0$表示。根据$y=x^2$可算出y的最大值是$15^2=225$，可以用8位二进制数$Y=Y_7Y_6Y_5Y_4Y_3Y_2Y_1Y_0$表示。由此可列出$Y=B^2$即$y=x^2$的真值表。

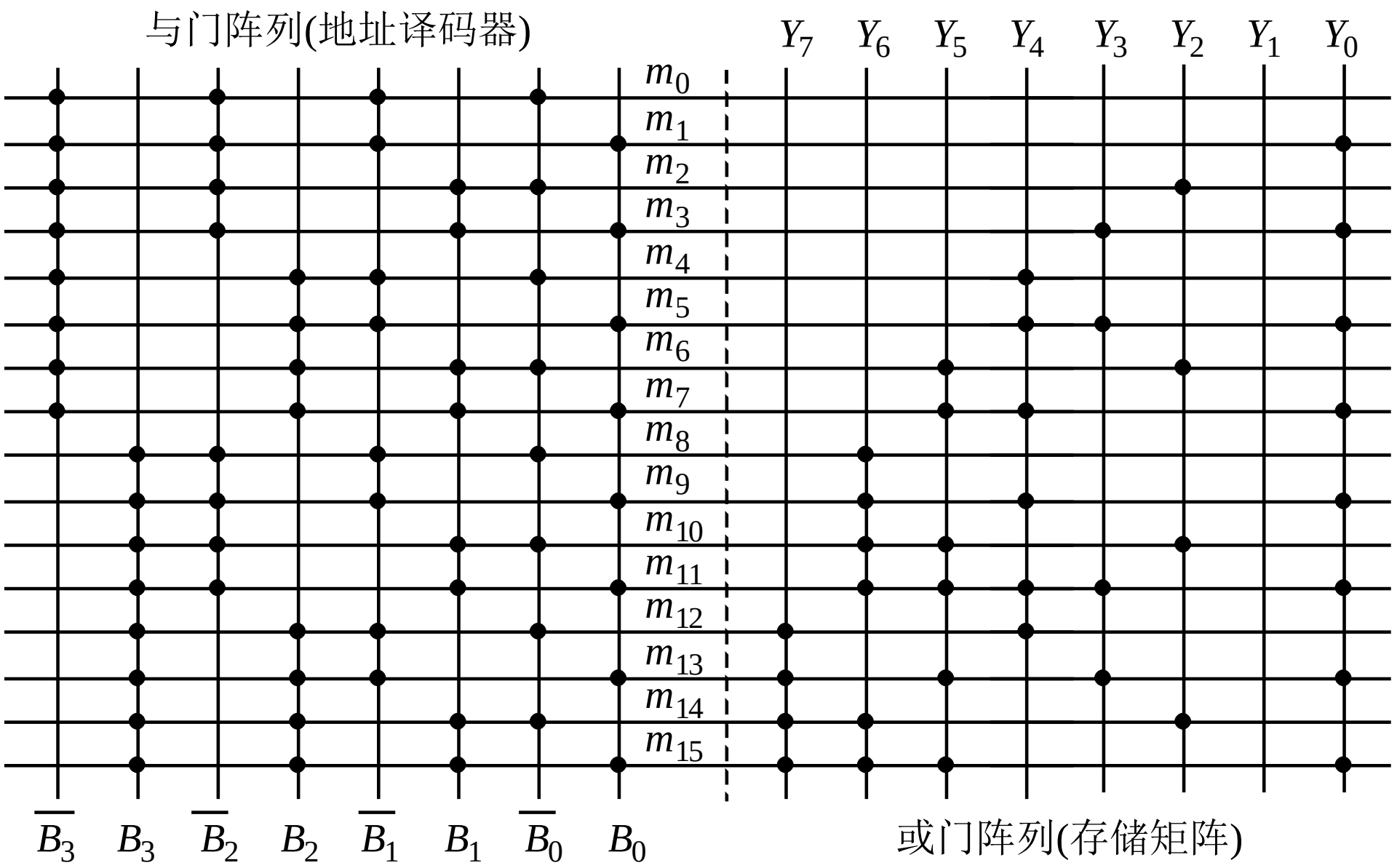
真值表

输入				输出								注
B_3	B_2	B_1	B_0	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0	十进制数
0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0	0	0	0	1	1
0	0	1	0	0	0	0	0	0	1	0	0	4
0	0	1	1	0	0	0	0	1	0	0	1	9
0	1	0	0	0	0	0	1	0	0	0	0	16
0	1	0	1	0	0	0	1	1	0	0	1	25
0	1	1	0	0	0	1	0	0	1	0	1	36
0	1	1	1	0	0	1	1	0	0	0	1	49
1	0	0	0	0	1	0	0	0	0	0	0	64
1	0	0	1	0	1	0	1	0	0	0	1	81
1	0	1	0	0	1	1	0	0	1	0	0	100
1	0	1	1	0	1	1	1	1	0	0	1	121
1	1	0	0	1	0	0	1	0	0	0	0	144
1	1	0	1	1	0	1	0	1	0	0	1	169
1	1	1	0	1	1	0	0	0	1	0	0	196
1	1	1	1	1	1	1	0	0	0	0	1	225

逻辑表达式

$$
\begin{cases}
Y_7 = \sum m(12,13,14,15) \\
Y_6 = \sum m(8,9,10,11,14,15) \\
Y_5 = \sum m(6,7,10,11,13,15) \\
Y_4 = \sum m(4,5,7,9,11,12) \\
Y_3 = \sum m(3,5,11,13) \\
Y_2 = \sum m(2,6,10,14) \\
Y_1 = 0 \\
Y_0 = \sum m(1,3,5,7,9,11,13,15)
\end{cases}
$$

阵列图

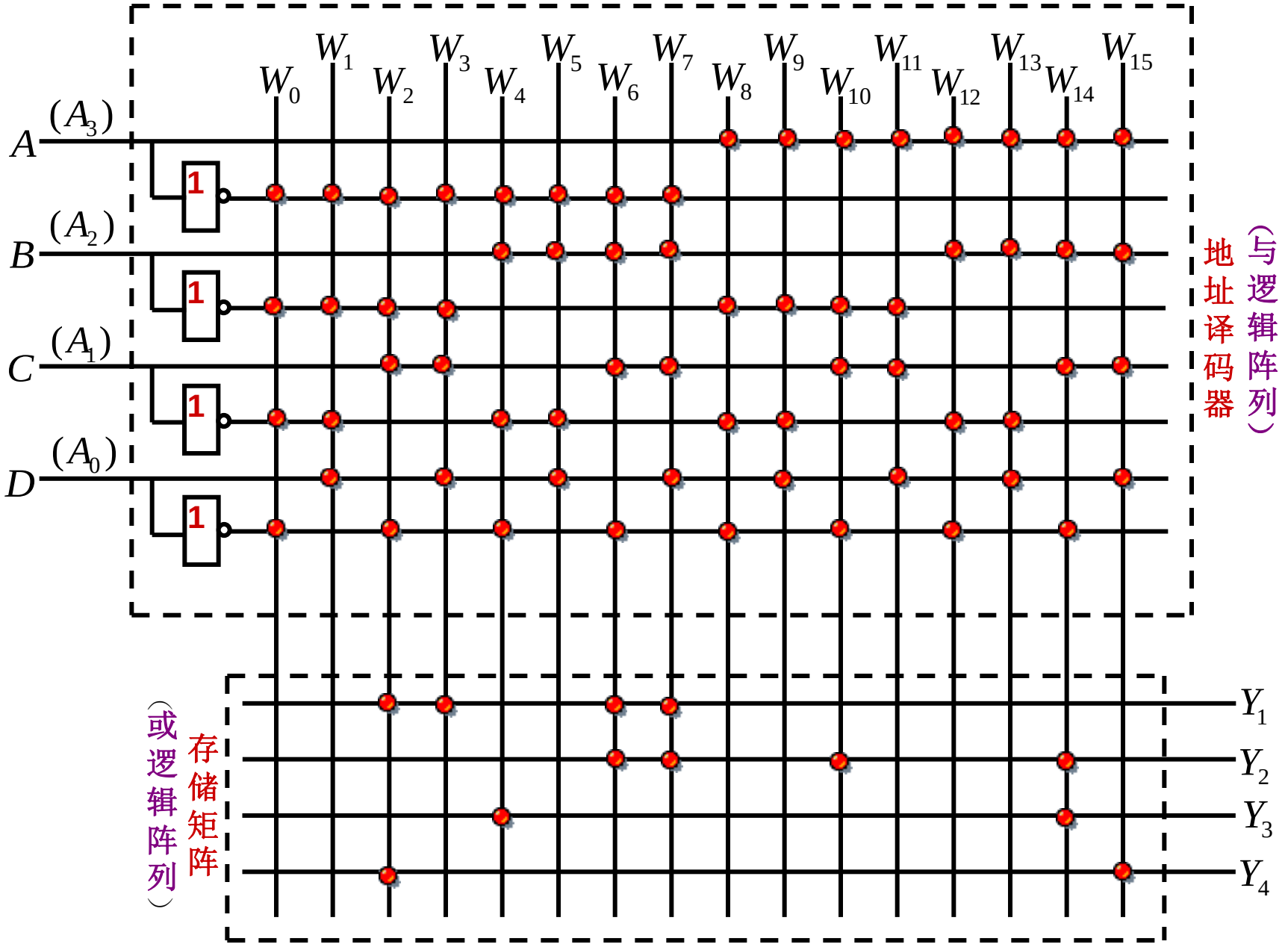

与门阵列(地址译码器)
Y_7 Y_6 Y_5 Y_4 Y_3 Y_2 Y_1 Y_0
m_0 m_1 m_2 m_3 m_4 m_5 m_6 m_7 m_8 m_9 m_{10} m_{11} m_{12} m_{13} m_{14} m_{15}
$\overline{B_3}$ B_3 $\overline{B_2}$ B_2 $\overline{B_1}$ B_1 $\overline{B_0}$ B_0
或门阵列(存储矩阵)

[例7.5.1] 试用ROM产生如下的一组多输出逻辑函数。

$$\begin{cases} Y_1 = A'BC + A'B'C \\ Y_2 = AB'CD' + BCD' + A'BCD \\ Y_3 = ABCD' + A'BC'D' \\ Y_4 = A'B'CD' + ABCD \end{cases}$$

解：将输出函数展开成标准与-或表达式：

$$\begin{cases} Y_1 = A'BCD' + A'BCD + A'B'CD' + A'B'CD \\ Y_2 = AB'CD' + A'BCD' + ABCD' + A'BCD \\ Y_3 = ABCD' + A'BC'D' \\ Y_4 = A'B'CD' + ABCD \end{cases} \quad \begin{cases} Y_1 = m_2 + m_3 + m_6 + m_7 \\ Y_2 = m_6 + m_7 + m_{10} + m_{14} \\ Y_3 = m_4 + m_{14} \\ Y_4 = m_2 + m_{15} \end{cases}$$



例7.5.1的ROM点阵图

作业

- P384　7.8　7.9　7.10　7.11　7.14